

PE DV 算子运行及分析方法

1. 账号申请

sipu_sw代码仓位置：https://gitlabsoft.siorigin.com/arch/sipu_sw

若无权限，请联系  冷冰  郑宏运 申请

需要申请ETX账号，若无权限，请联系IT申请

ETX地址：[Exceed TurboX Dashboard](#)

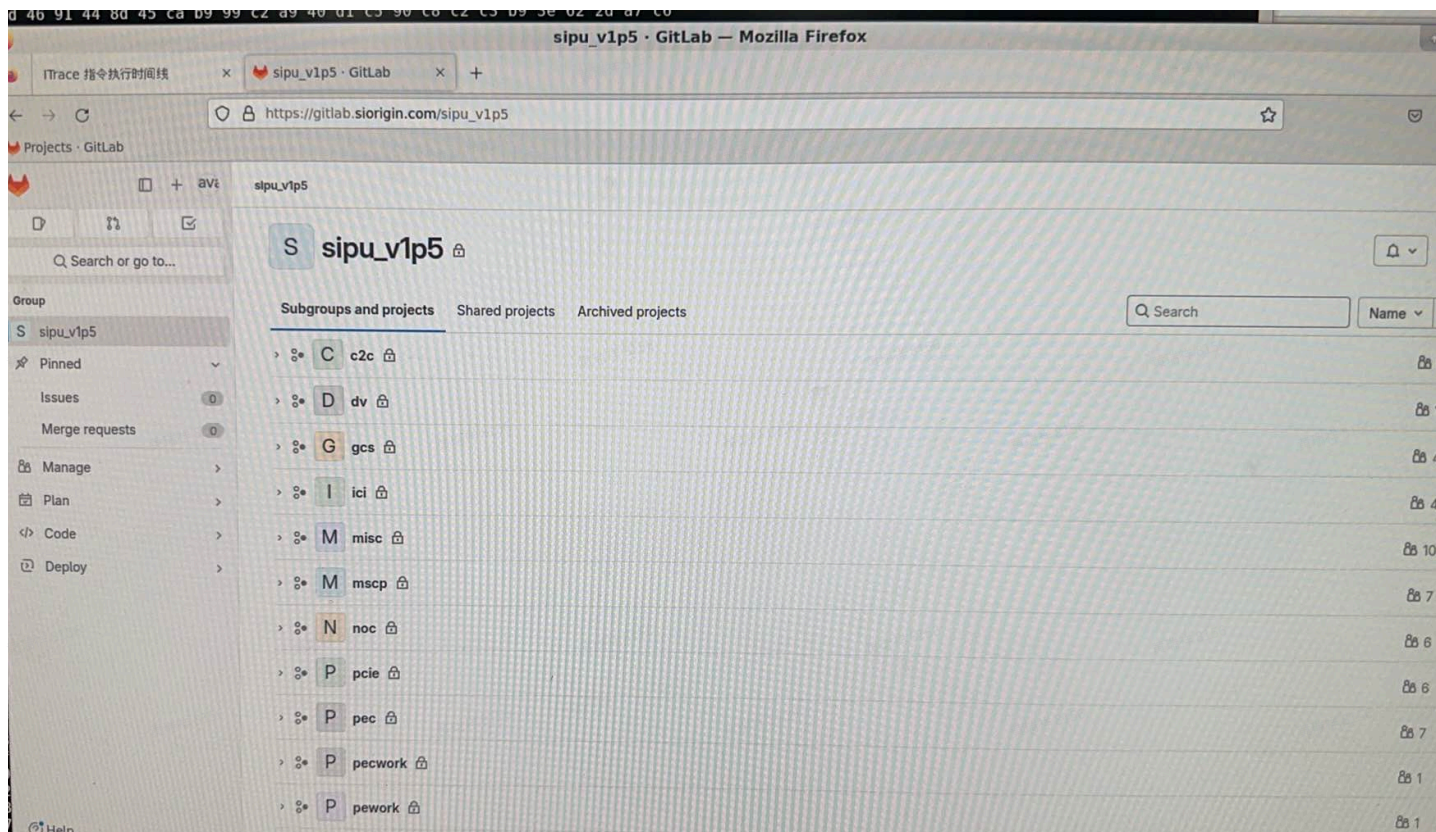
硬件部门Gitlab地址：<https://gitlab.siorigin.com>（浏览器若无法打开，直接terminal输入firefox&）

若无权限，请联系  春鹏 申请

账号：中文拼音（比如：zhangyawen）初始密码：ZJXJzjxj123!

密码修改地址：<http://10.192.2.50/index.php>

内网代码仓位置：https://gitlab.siorigin.com/sipu_v1p5



2. Vector dump

若想要自己生成DV可执行kernel，需要产生vectordump文件，通过sipu_sw代码仓产生 sipw_sw编译方法参考 [SiPU架构&软件协同开发环境](#) 比如在架构服务器上

代码块

```
1 mkdir proj && cd proj
2 # setup python virtual env
3 python3 -m venv py3
4 source py3/bin/activate
5
6 # clone代码仓
7 git clone https://gitlabsoft.siorigin.com/arch/sipu_sw.git
8 cd sipu_sw
9 git pull && git submodule update --init --recursive
10 python make/build_sipu_repo.py rel --skip_list qemu
```

等待编译完成后，生成vector dump

代码块

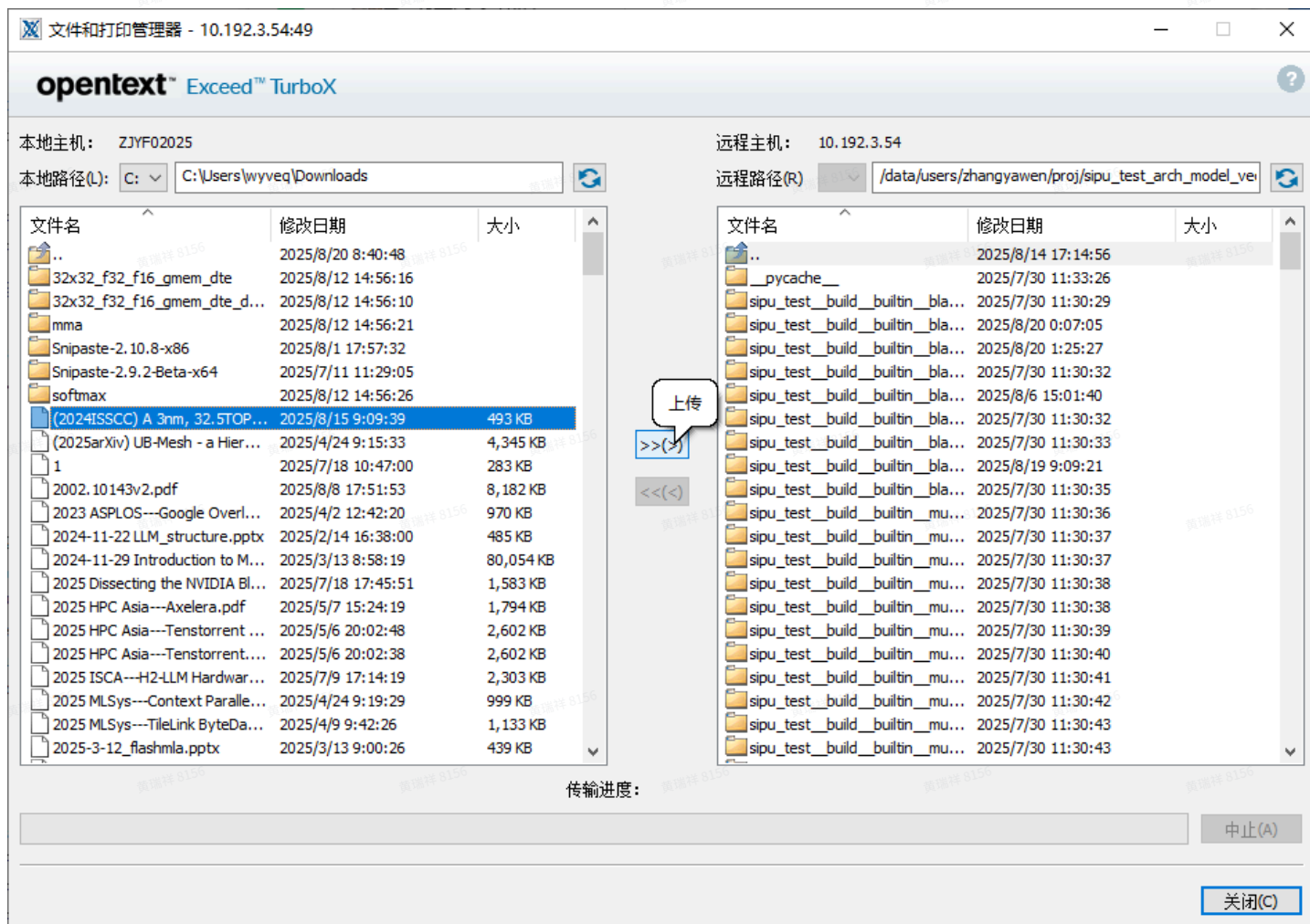
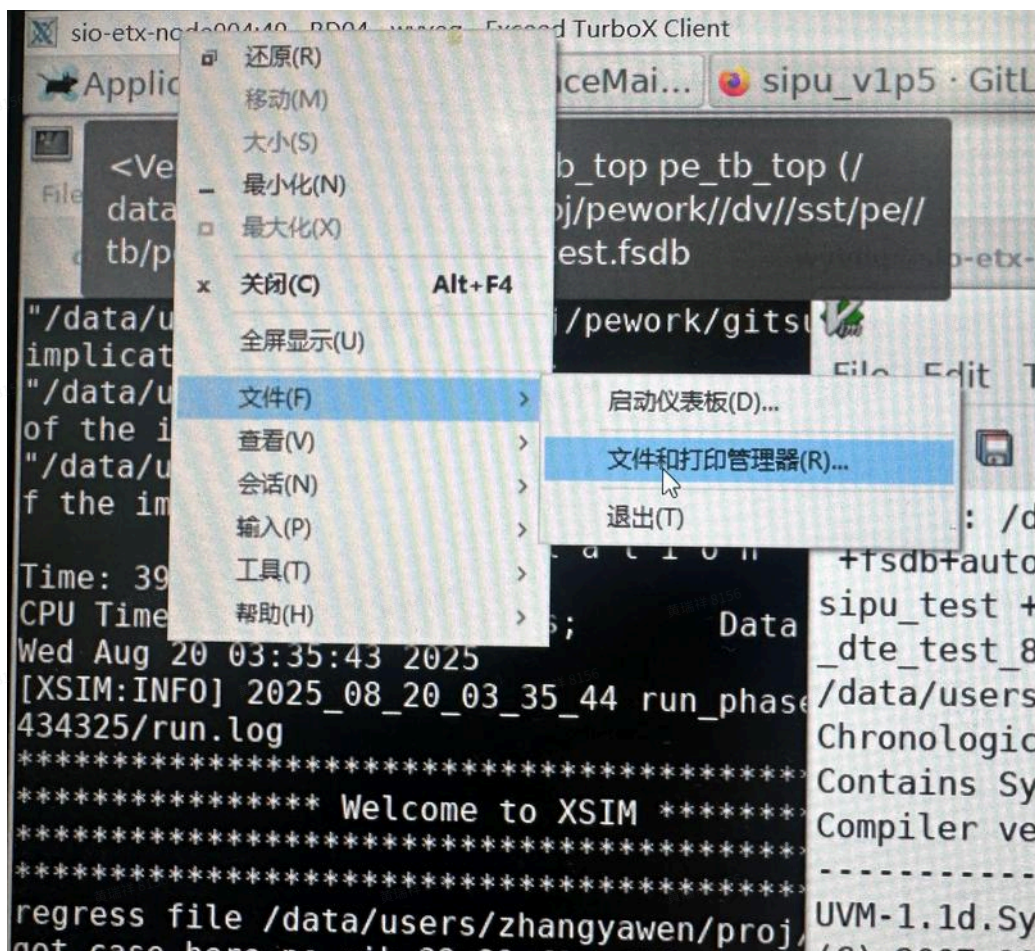
```
1 ./ci_test/sikernel/run_arch_model_vector_dump.sh --mp 64
2 ./ci_test/sipu_test/run_arch_model_vector_dump.sh --mp 64
```

比如生成sipu_test内kernel，可修改 `ci_test/sipu_test/arch_model/vec_test_list` 里面kernel列表，只生成所需kernel

```
• (py3) wyveg@siso-arch-node003:~/sipu_sw$ ./ci_test/sipu_test/run_arch_model_vector_dump.sh --mp 64
test list order random seed is 1755618257
pytest -v sipu_test_arch_model_vector_dump/tests.py -n 64
===== test session starts =====
platform linux -- Python 3.10.12, pytest-8.3.5, pluggy-1.5.0 -- /data/users/zhangyawan/proj/py3/bin/python3
cachedir: .pytest_cache
metadata: {'Python': '3.10.12', 'Platform': 'Linux-6.5.0-18-generic-x86_64-with-glibc2.35', 'Packages': {'pytest': '8.3.5', 'pluggy': '1.5.0'}, 'Plugins': {'xdist': '3.6.1',
'timeout': '2.4.0', 'metadata': '3.1.1', 'html': '4.1.1'}}
rootdir: /data/users/zhangyawan/sipu_sw
plugins: xdist-3.6.1, timeout-2.4.0, metadata-3.1.1, html-4.1.1
64 workers [10 items]
scheduling tests via LoadScheduling

sipu_test_arch_model_vector_dump/tests.py::test_sipu_test_build_builtin_blas_mma_32x32_f32_f16_gmem_dte_doublebufferN_32x32_f32_f16_gmem_dte_doublebufferN PASSED
sipu_test_arch_model_vector_dump/tests.py::test_sipu_test_build_builtin_blas_mma_32x32_f32_f16_gmem_dte_32x32_f32_f16_gmem_dte PASSED
sipu_test_arch_model_vector_dump/tests.py::test_sipu_test_build_builtin_blas_silu_and_mul_f32_silu_and_mul_f32 PASSED
sipu_test_arch_model_vector_dump/tests.py::test_sipu_test_build_builtin_blas_mma_32x32_f32_f16_smem_unroll_32x32_f32_f16_smem_unroll PASSED
sipu_test_arch_model_vector_dump/tests.py::test_sipu_test_build_builtin_blas_mma_32x32_f32_f16_simple_32x32_f32_f16_simple PASSED
sipu_test_arch_model_vector_dump/tests.py::test_sipu_test_build_builtin_blas_softmax_softmax_f32_softmax_f32 PASSED
sipu_test_arch_model_vector_dump/tests.py::test_sipu_test_build_builtin_blas_sigmoid_sigmoid_f32_sigmoid_f32 PASSED
sipu_test_arch_model_vector_dump/tests.py::test_sipu_test_build_builtin_blas_softmax_softmax_tile_softmax_tile PASSED
sipu_test_arch_model_vector_dump/tests.py::test_sipu_test_build_builtin_blas_mma_32x32_f32_f16_smem_simple_32x32_f32_f16_smem_simple PASSED
[gw1] [ 10%] PASSED sipu_test_arch_model_vector_dump/tests.py::test_sipu_test_build_builtin_blas_sigmoid_sigmoid_f32_sigmoid_f32
[gw0] [ 20%] PASSED sipu_test_arch_model_vector_dump/tests.py::test_sipu_test_build_builtin_blas_softmax_softmax_tile_softmax_tile
[gw5] [ 30%] PASSED sipu_test_arch_model_vector_dump/tests.py::test_sipu_test_build_builtin_blas_softmax_softmax_f32_softmax_f32
[gw9] [ 40%] PASSED sipu_test_arch_model_vector_dump/tests.py::test_sipu_test_build_builtin_blas_mma_32x32_f32_f16_smem_simple_32x32_f32_f16_smem_simple
[gw8] [ 50%] PASSED sipu_test_arch_model_vector_dump/tests.py::test_sipu_test_build_builtin_blas_mma_32x32_f32_f16_simple_32x32_f32_f16_simple
[gw4] [ 60%] PASSED sipu_test_arch_model_vector_dump/tests.py::test_sipu_test_build_builtin_blas_mma_32x32_f32_f16_simple_32x32_f32_f16_simple
[gw3] [ 70%] PASSED sipu_test_arch_model_vector_dump/tests.py::test_sipu_test_build_builtin_blas_mma_32x32_f32_f16_smem_unroll_32x32_f32_f16_smem_unroll
[gw7] [ 80%] PASSED sipu_test_arch_model_vector_dump/tests.py::test_sipu_test_build_builtin_blas_silu_and_mul_silu_and_mul_f32_silu_and_mul_f32
[gw6] [ 90%] PASSED sipu_test_arch_model_vector_dump/tests.py::test_sipu_test_build_builtin_blas_mma_32x32_f32_f16_gmem_dte_doublebufferN_32x32_f32_f16_gmem_dte_doublebufferN
[gw2] [100%] PASSED sipu_test_arch_model_vector_dump/tests.py::test_sipu_test_build_builtin_blas_mma_32x32_f32_f16_gmem_dte_32x32_f32_f16_gmem_dte
```

需要特别注意一点，若采用纯汇编写的kernel，可能存在编译器没有识别到tile reg个数问题，需要确认kernel内部的vector_dump中的ccs_bd_vector和ccs_bd_vector_x_x第14行是否不为0，若为0则说明编译器没有正确识别汇编的寄存器个数，需要手动修改！



3. DV 环境配置

所有操作均在ETX内网完成！！

3.1 git代码下载

需要先配置github的ssh key

代码块

```
1  git clone ssh://git@gitlab.siorigin.com:43101/sipu_v1p5/pework/pework.git -b dev
2  cd pework
3  git submodule update --init --recursive
4  cd gitsubm/de_common
5  git checkout dev
6  git pull && git submodule update --init --recursive
7  cd ../pec
8  git checkout dev
9  git pull && git submodule update --init --recursive
10 cd ../v_cmn
11 git checkout dev
12 git pull && git submodule update --init --recursive
```

因为当前PE在修改，可能DV环境不是很稳定，可咨询  马彩霞 找可用的commit

我采用的是0812版本，但是建议用最新的版本，尤其是包含PE修改的版本

PEC的commit: ef403c5ce48dcda14a118defb99f527331f66a438

v_cmn的commit: adc4db4e556f39ec3d73cb8f3ce88a5853ac34ab

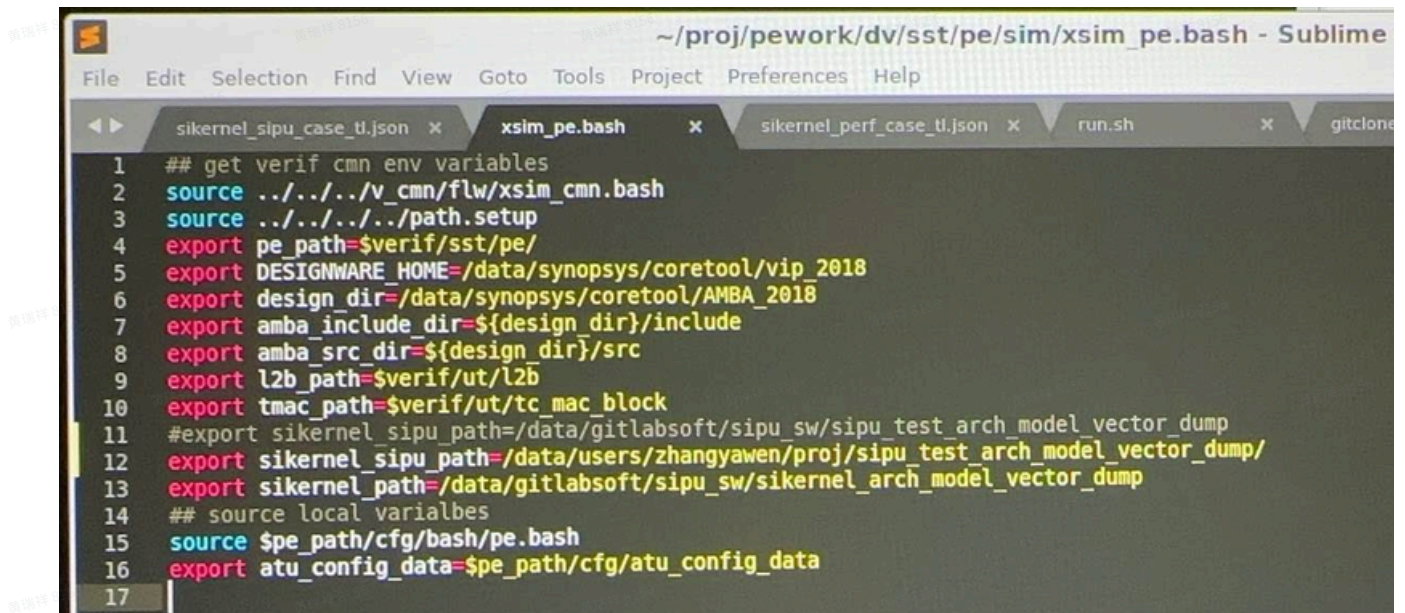
pework的commit: 6b6f72e4bc4c480c0322284cc8b32d8c371aa760

3.2 配置kernel path

配置 xsim_pe.bash 文件在 `dv/sst/pe/sim` 目录下，每次运行dv前，需要执行

代码块

```
1  source xsim_pe.bash
```



```
1 ## get verif cmn env variables
2 source ../../../../v_cmn/flw/xsim_cmn.bash
3 source ../../../../path.setup
4 export pe_path=$verif/sst/pe/
5 export DESIGNWARE_HOME=/data/synopsys/coretool/vip_2018
6 export design_dir=/data/synopsys/coretool/AMBA_2018
7 export amba_include_dir=${design_dir}/include
8 export amba_src_dir=${design_dir}/src
9 export l2b_path=$verif/ut/l2b
10 export tmac_path=$verif/ut/tc_mac_block
11 #export sikernel_sipu_path=/data/gitlabsoft/sipu_sw/sipu_test_arch_model_vector_dump
12 export sikernel_sipu_path=/data/users/zhangyawen/proj/sipu_test_arch_model_vector_dump/
13 export sikernel_path=/data/gitlabsoft/sipu_sw/sikernel_arch_model_vector_dump
14 ## source local varialbes
15 source $pe_path/cfg/bash/pe.bash
16 export atu_config_data=$pe_path/cfg/atu_config_data
17
```

若需要跑自己生成的kernel，需要修改sikernel_sipu_path或者sikernel_path，比如写的kernel在sipu_test里面，即替换sikernel_sipu_path为上面对应的传输位置

代码块

```
1 export sikernel_sipu_path=/data/users/zhangyawen/proj/siputest_arch_model_vector_dump/
2 export sikernel_path=/data/gitlabsoft/sipu_sw/sikernel_arch_model_vector_dump
```

4. DV 运行

4.1 环境source

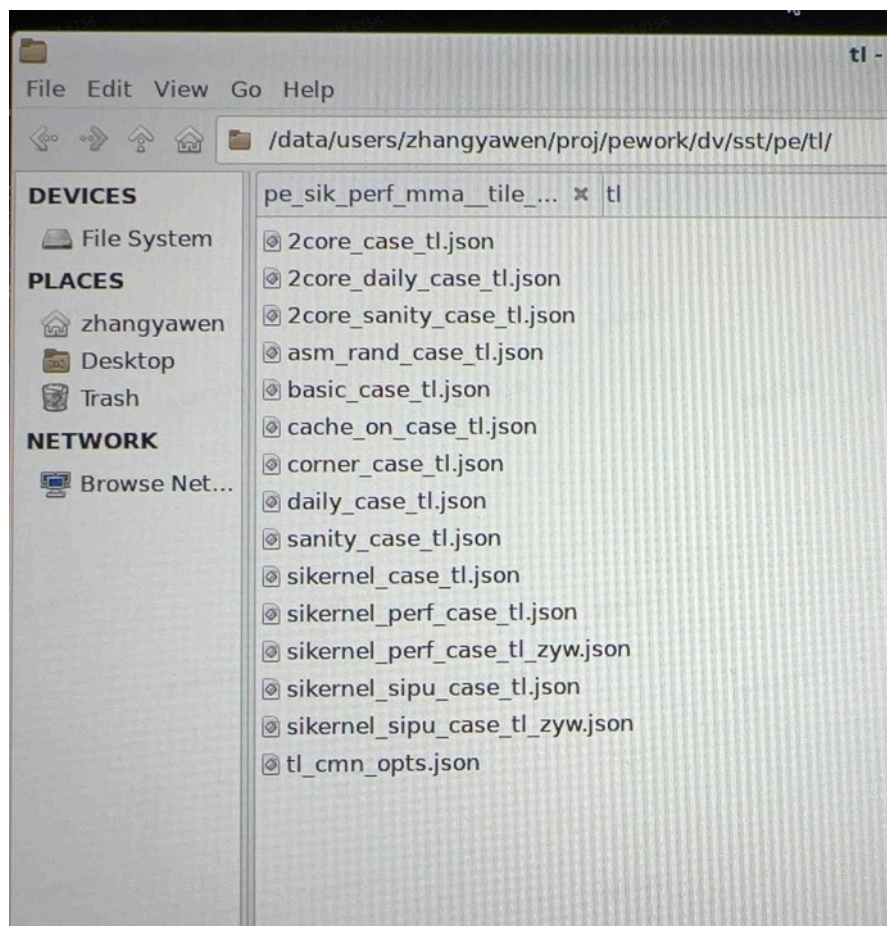
代码块

```
1 cd dv/sst/pe/sim
2 source xsim_pe.bash
```

4.2 TL配置

只能运行Test List (TL) 列表中存在的kernel，列表位置在 `pework/dv/sst/pe/tl/`

- sikernel_case_tl.json: sikernel中kernel
- sikernel_perf_case_tl.json: sikernel perf test中kernel
- sikernel_sipu_case_tl.json: sipu test中kernel



这里以执行sipu_test中的softmax_f32为例，打开

pework/dv/sst/pe/tl/sikernel_sipu_case_tl.json

```
~/proj/pework/dv/sst/pe/tl/sikernel_sipu_case_tl.json - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto Tools Project Preferences Help

sikernel_sipu_case_tl.json x xsim_pe.bash x sikernel_perf_case_tl.json x run.sh x gitclone.sh x softmax_f32_kernel.d x

1 {
2   "name": "sikernel sipu case tl.json", "type": "tl",
3   "$ref": ["$pe_path/tl/tl_cm_n_opts.json", ""],
4   "pe_sik 32x32 f32 f16 gmem dte test": {"type": "tc", "usr_run_opts": ["+UVM_TIMEOUT=\\"1000000ns,YES\\"
5   +sikernel test=builtin blas_mma_32x32 f32 f16 gmem dte_32x32 f32 f16 gmem dte +ilm boot +sipu test"], "uvm_tn": "pe_ccs_ilm_boot_test"},
6   "pe_sik 32x32 f32 f16 smem simple test": {"type": "tc", "usr_run_opts": ["+UVM_TIMEOUT=\\"1000000ns,YES\\"
7   +sikernel test=builtin blas_mma_32x32 f32 f16 smem simple_32x32 f32 f16 smem simple +ilm boot +sipu test"], "uvm_tn": "pe_ccs_ilm_boot_test"},
8   "pe_sik 32x32 f32 f16 smem unroll test": {"type": "tc", "usr_run_opts": ["+UVM_TIMEOUT=\\"1000000ns,YES\\"
9   +sikernel test=builtin blas_mma_32x32 f32 f16 smem unroll_32x32 f32 f16 smem unroll +ilm boot +sipu test"], "uvm_tn": "pe_ccs_ilm_boot_test"},
10  "pe_sik 32x32 f32 f16 gmem dte doublebufferN test": {"type": "tc", "usr_run_opts": ["+UVM_TIMEOUT=\\"1000000ns,YES\\"
11  +sikernel test=builtin blas_mma_32x32 f32 f16 gmem dte doublebufferN_32x32 f32 f16 gmem dte doublebufferN +ilm boot +sipu test +force_l2c_prefetch
12  +force_dcach_en +big_stress_b +big_stress_r +null_cycles_r=150 +null_cycles_b=50 +ITRACE_DEBUG_ENABLE +no_inter +in_order"], "uvm_tn":
13  "pe_ccs_ilm_boot_test"},
14  "pe_sik softmax f32 test": {"type": "tc", "usr_run_opts": ["+UVM_TIMEOUT=\\"1000000ns,YES\\" +sikernel test=builtin blas_softmax_softmax_f32_softmax_f32
15  +ilm boot +sipu test +ilm boot +sipu test +force_l2c_prefetch +force_dcach_en +big_stress_b +big_stress_r +null_cycles_r=150 +null_cycles_b=50
16  +ITRACE_DEBUG_ENABLE +no_inter +in_order +disable_inst_checker"], "uvm_tn": "pe_ccs_ilm_boot_test"},
17
18
19
20
21
22  "pe_sik softmax tile test": {"type": "tc", "usr_run_opts": ["+UVM_TIMEOUT=\\"1000000ns,YES\\"
23  +sikernel test=builtin blas_softmax_softmax_tile_softmax_tile +ilm boot +sipu test"], "uvm_tn": "pe_ccs_ilm_boot_test"},
24
25
26
27
28
29
30
31
32
```

- 最上面name是tl的名称 sikernel_sipu_case_tl

- 执行算子对应的名称为 `pe_sik_softmax_f32_test`

可以看到后面有一堆的参数，这是运行dv的可选配置参数，建议配置如下，如有问题可咨询

玲利 仲玲利

大类	配置	方法
Prefetch	L2C prefetch	+forece_l2c_prefetch
	Dcache prefetch	+force_dcache_en
DRAM	固定DRAM latency	+null_cycles_r=150 +null_cycTes_b=50
	DRAM不开interleave	+no_inter
	DRAM不开out of order	+in_order
		+big_stress_b +big_stress_r
Check	禁用check，softmax需要（可选）	+disable_inst_checker
Exception	去除TREG_OOB的tcore exception（可选，也可以修改vectordump bd规避）	+force_tregsize
ITRACE	增加debug itrace信息（可选）	+ITRACE_DEBUG_ENABLE

代码块

```
1  "pe_sik_softmax_f32_test" : {"type":"tc", "usr_run_opts":  
  ["+UVM_TIMEOUT=\"1000000ns,YES\""  
  +sikerne1_test=builtn_blas_ssoftmax_softmax_f32_softmax_f32  
2  +ilm boot +sipu_test +ilm_boot +sipu_test +force_l2c_prefetoth  
  +force_dcache_en +big_stress_b +big_stress_r_ +null_cycles_r=150  
  +null_cycles_b=50  
3  +ITRACE_DEBUG_ENABLE +no_inter +in_order +disable_inst_checker"],  
  "uvm_tn":"pe_ccs_ilm_boot_test"}
```

4.3 编译&运行

若第一次运行，需要arun编译DV环境

- `-regn pe_2core`：执行两个PE core
- `-tl`：对应Test List (TL) 列表名称
- `-tc`：对应kernel名称

- `-sim_out_dir output`：指定仿真结果输出目录，所有 log、波形、结果文件会写到 output/ 目录里
- `arun` (all run)：dut + testbench 都编译
- `vrn` (verif run)：dut 不编译，testbench 编译
- `ncrun` (not compile run)：dut + testbench 都不编译，当只是修改 tl 的 json 文件（增加用例或者修改用例），可以使用这种模式

代码块

```
1 xsim -mode arun -regn pe_2core -tl skernel_sipu_case_tl -tc  
  pe_sik_kernel_softmax_f32_test -sim_out_dir output
```

DV 环境仅需要编译一次即可，第二次运行执行 `ncrun`

代码块

```
1 xsim -mode ncrun -regn pe_2core -tl skernel_sipu_case_tl -tc  
  pe_sik_kernel_softmax_f32_test -sim_out_dir output
```

DV 运行中出现问题，请联系  仲玲利  马彩霞

结果的性能问题，若分析不清楚可联系  翟少敏  张雅文  张世界  尔荟东

5. 结果查看

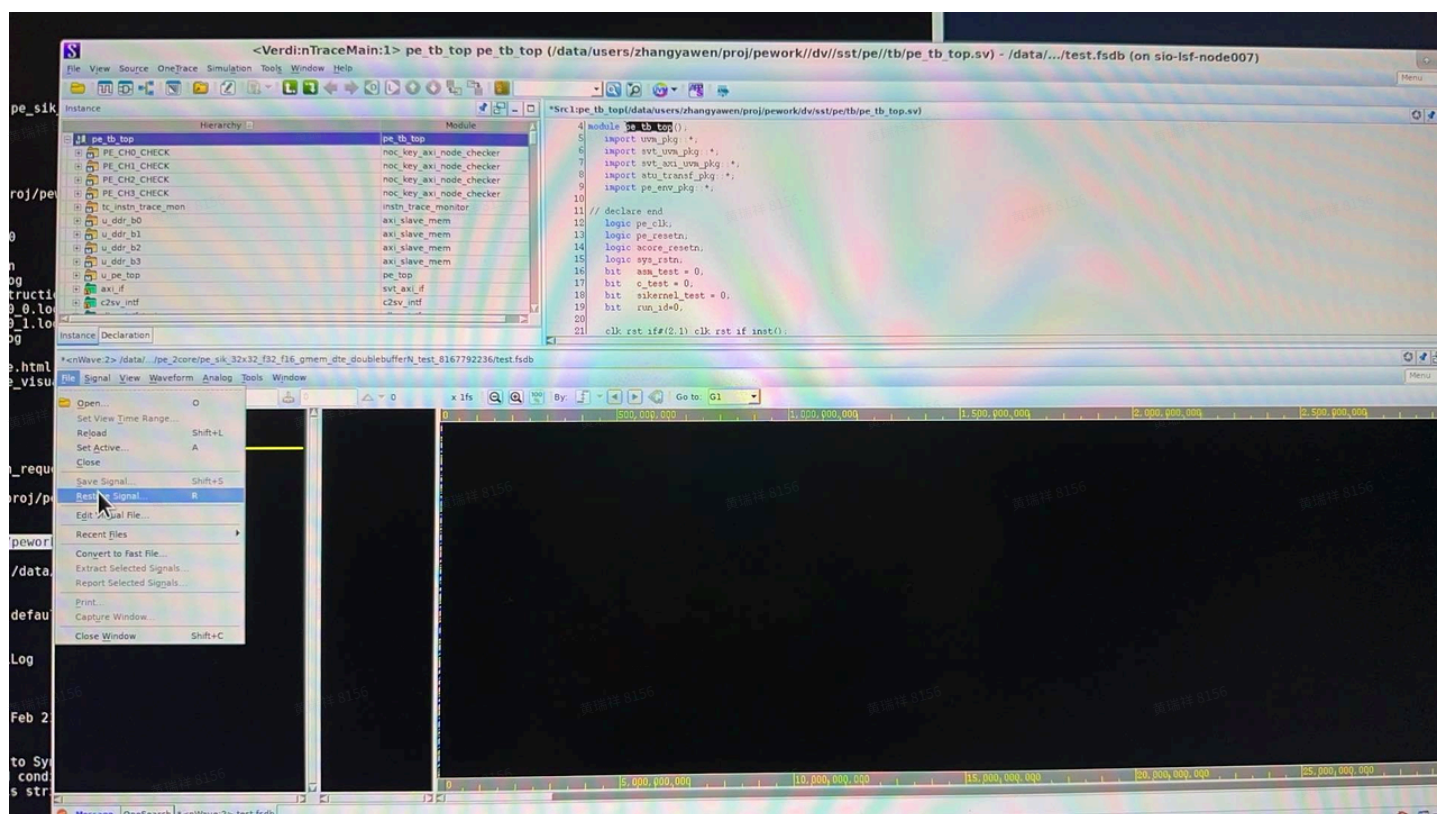
执行结束后，首先会生成一个 pass or fail 打印

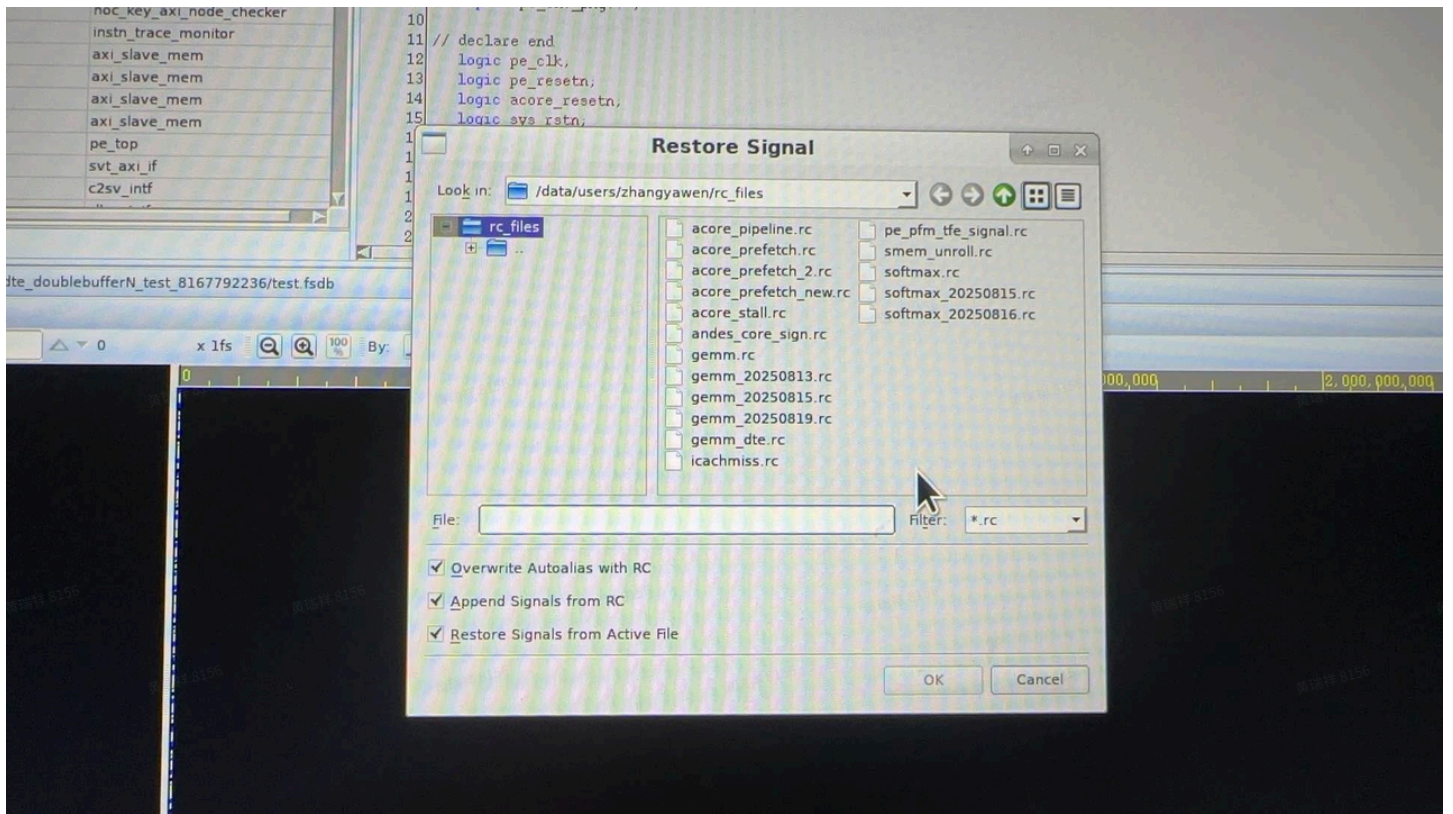
波形通过运行test.fsdb查看

代码块

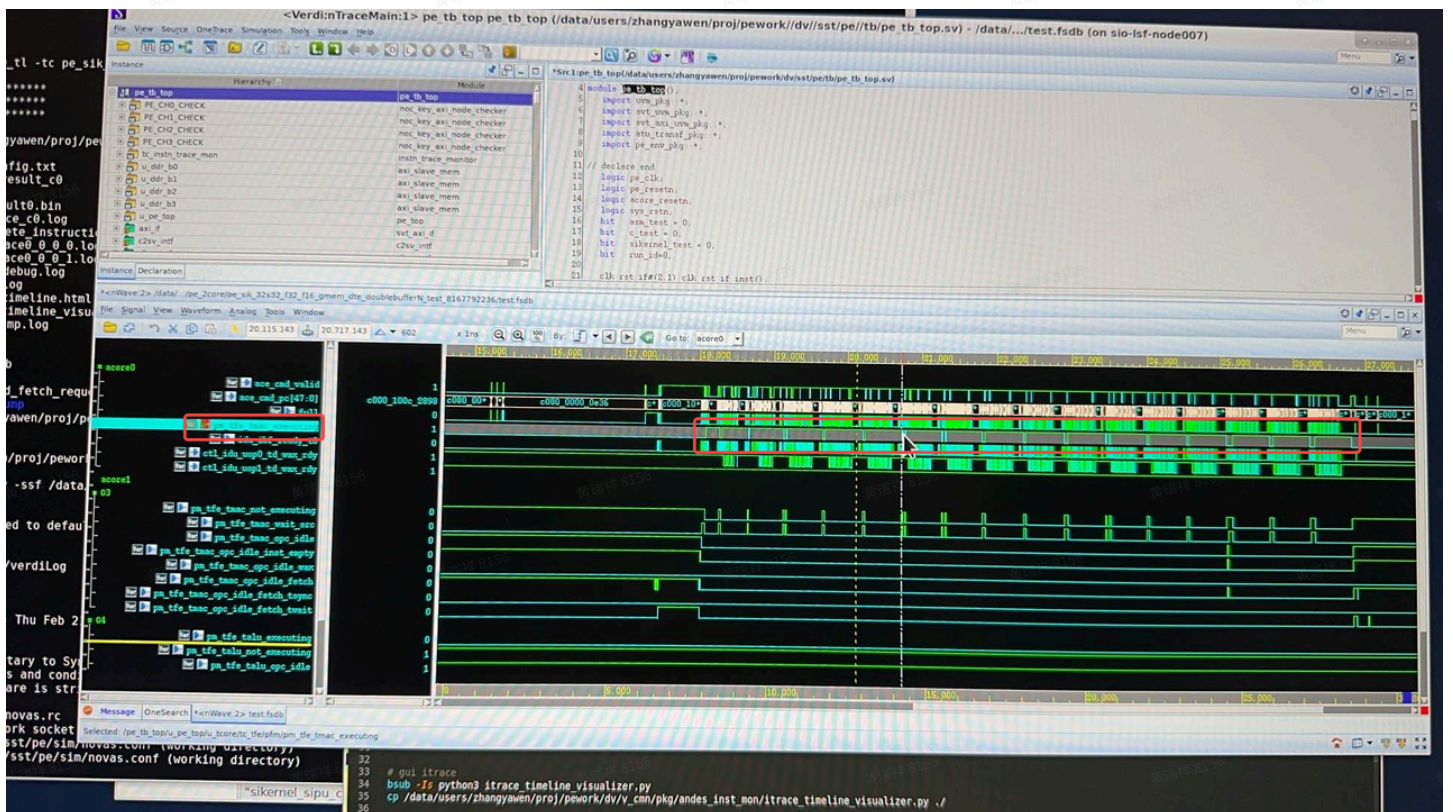
```
1 bsub -Is verdi -simflow -ssf  
output/pe_1core/pe_sik_kernel_launch_test/test.fsdb &
```

会打开verdi，需要rc文件查看波形关键信号，若需要可去目录 `/data/users/zhangyawan/rc_files` 获取

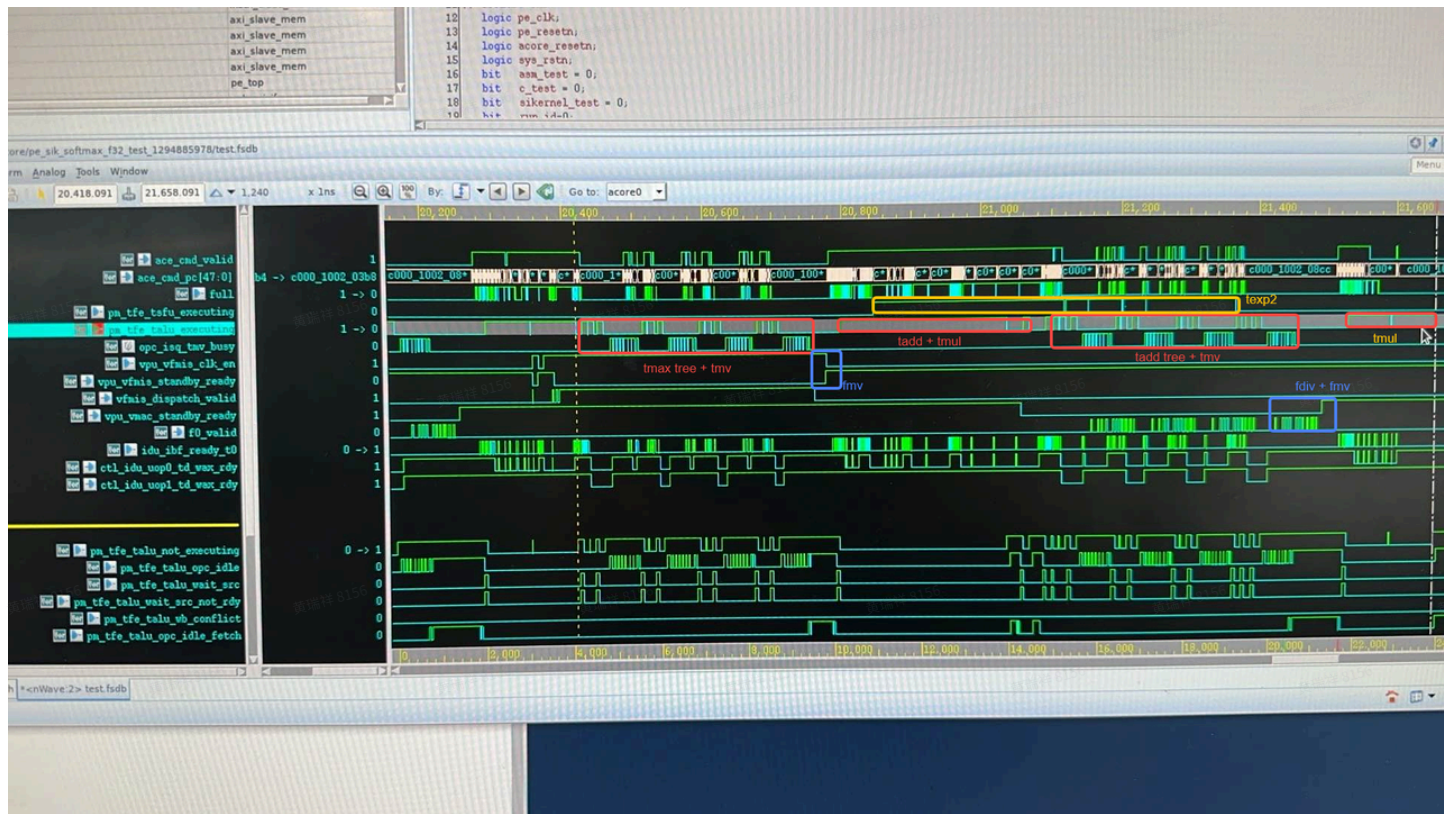




GEMM可采用最新的 `gemm_xx.rc`，主要看 `pm_tfe_tmac_executing` 信号，可以看到mac基本被打满



Softmax可采用最新的 `softmax_xx.rc`，主要看 `pm_tfe_tsfu_executing`、`pm_tfe_talu_executing`、`opc_isq_tmv_busy` 信号，分别对应TSFU、TALU、TMV的忙闲



参考：

冷冰写的[国DV仿真环境使用](#)

坑爹的DV测试史 [国DV perf 算子性能分析](#)